

Exception Handling

MUSANINYANGE M.Larisse



What is exception ?

- **In Java, Exception** is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions.
- Exceptions can be caught and handled by the program. When an exception occurs within a method, it creates an object. This object is called the exception object. It contains information about the exception, such as the name and description of the exception and the state of the program when the exception occurred.



Exception reasons

Major reasons why an exception Occurs

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out-of-disk memory)
- Code errors
- Opening an unavailable file



Errors vs Exception

- **Error:** An Error indicates a serious problem that a reasonable application should not try to catch.
- **Exception:** Exception indicates conditions that a reasonable application might try to catch.



Exception Hierarchy

- All exception and error types are subclasses of the class **Throwable**, which is the base class of the hierarchy.
- One branch is headed by **Exception**. This class is used for exceptional conditions that user programs should catch. `NullPointerException` is an example of such an exception.
- Another branch, **Error** is used by the Java run-time system(JVM) to indicate errors having to do with the run-time environment itself(JRE). `StackOverflowError` is an example of such an error.



Advantages of proper exception handling in Java

- **Ensures the Continuity of the Program:**an unhandled exception would cause the program to terminate abruptly, which can lead to data loss & other issues. With proper exception handling, the program can continue to execute and provide a more stable user experience.
- **Enhances the Robustness of the Program:**Exception handling allows for the program to anticipate and recover from errors, thus making the program more robust and resistant to unexpected conditions. By catching and handling exceptions, the program can continue to execute and provide a more stable user experience.
-



Advantages of proper exception handling in Java

- **Improves the Readability & Maintainability of the Code:** Proper exception handling also improves the readability and maintainability of the code. By catching and handling exceptions, the program can provide clear error messages that accurately describe the error and provide information on how to resolve the issue. This makes it easier for developers to understand and modify the code in the future. Additionally, by providing detailed error messages, proper exception handling allows for more accurate error reporting, which is essential for debugging and troubleshooting purposes



Advantages of proper exception handling in Java

- **Allows for more Accurate Error Reporting:** Exception handling allows the program to catch & report errors in a more accurate & detailed manner, providing valuable information to developers for debugging and troubleshooting purposes.
- **Facilitates Debugging and Troubleshooting:** by providing detailed error messages and stack traces, exception handling allows developers to quickly identify and resolve issues, reducing the amount of time and resources required for debugging.
- **Improves the Security of the Program:** by catching and handling exceptions, the program can prevent sensitive information, such as passwords and personal data, from being displayed to the user or logged-in error messages.
- **Provides a Better user Experience:** Proper exception handling allows the program to anticipate and recover from errors, providing a more stable user experience

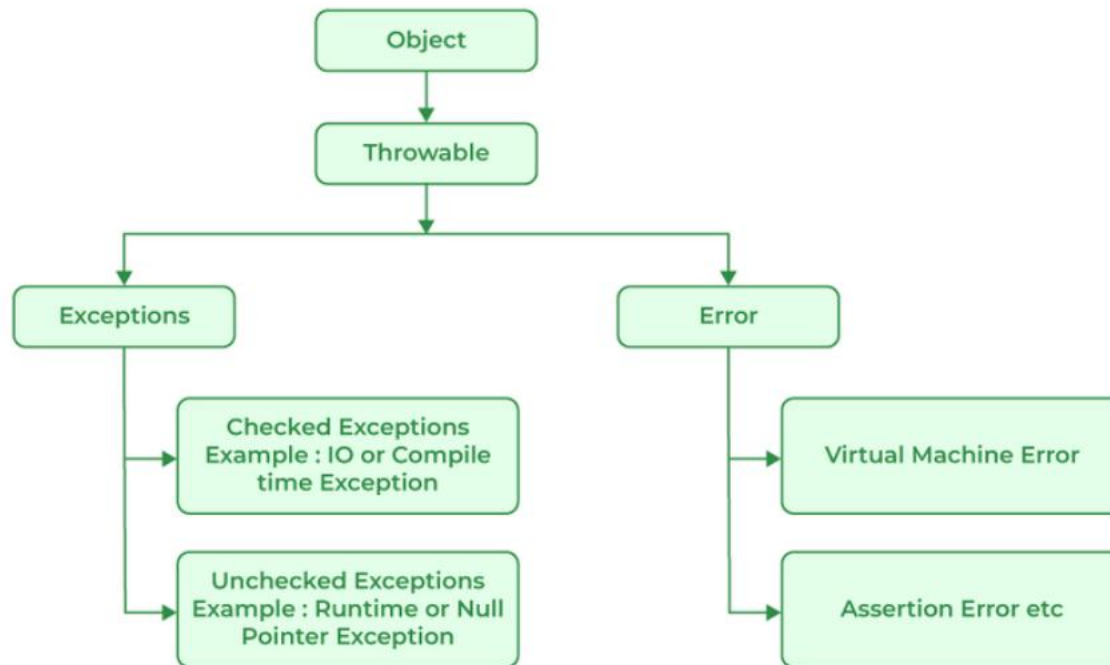


Best Practice

- **Catch only specific exceptions (avoid catching generic Exception).**
- **Always clean up resources using finally or try-with-resources.**
- **Use custom exceptions for meaningful error messages.**

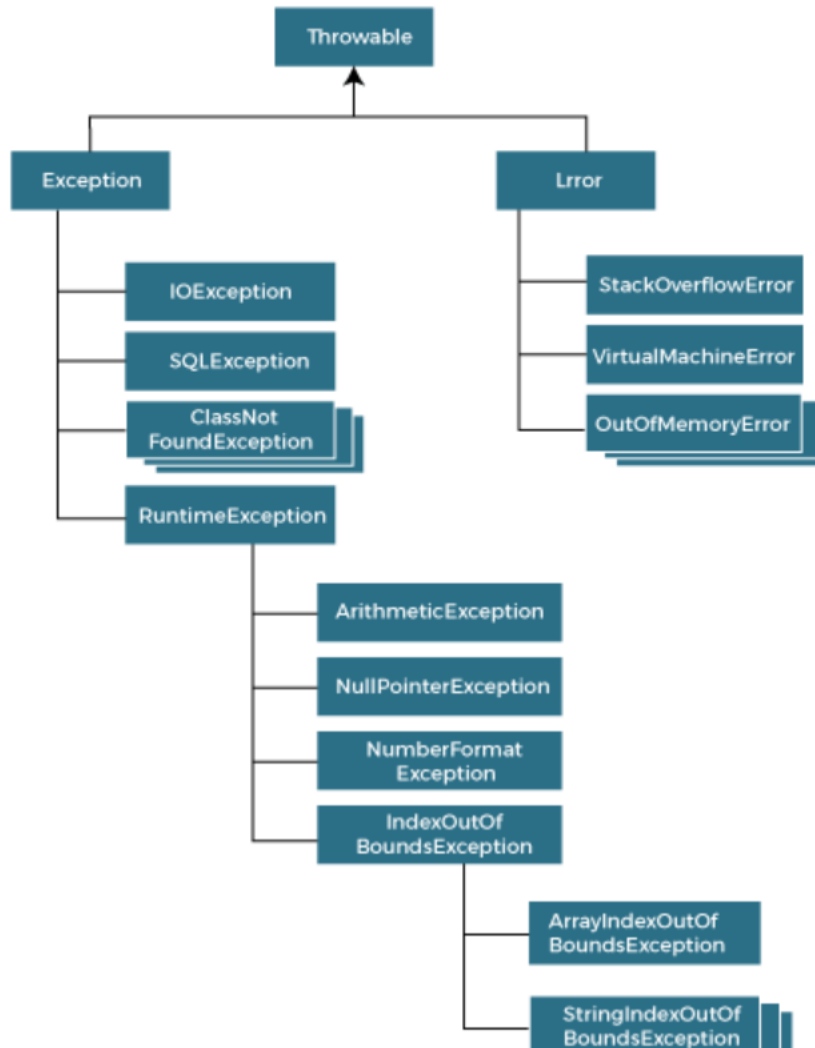


Exception Hierarchy

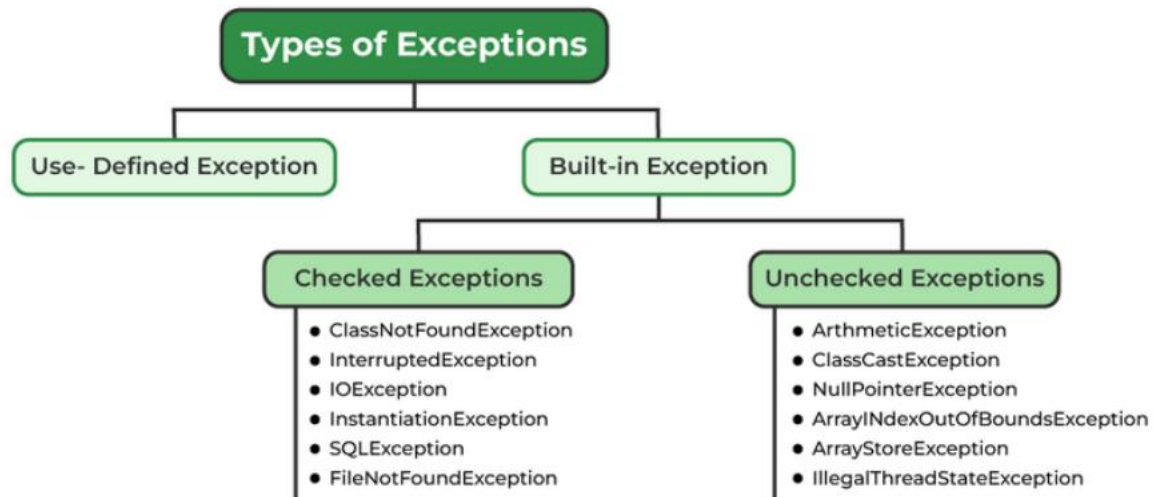


Java Exception Hierarchy

Exception Hierarchy



Types of exceptions



Categories of exceptions

Exceptions can be categorized in two ways:

- **Built-in Exceptions**
 - Checked Exception
 - Unchecked Exception
- **User-Defined Exceptions**



Built- in exceptions

Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations.

- **Checked Exceptions:** Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler.
- **Unchecked Exceptions:** The unchecked exceptions are just opposite to the checked exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error.



User- defined exceptions

- Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called '**user-defined Exceptions**'.



How JVM handles the exception

- **Default Exception Handling:** Whenever inside a method, if an exception has occurred, the method creates an Object known as an Exception Object and hands it off to the run-time system(JVM). The exception object contains the name and description of the exception and the current state of the program where the exception has occurred. Creating the Exception Object and handling it in the run-time system is called **throwing an Exception**. There might be a list of the methods that had been called to get to the method where an exception occurred. This ordered list of methods is called **Call Stack**. Now the following procedure will happen.
- The run-time system searches the call stack to find the method that contains a block of code that can handle the occurred exception. The block of the code is called an **Exception handler**.



How JVM handles the exception

- The run-time system starts searching from the method in which the exception occurred and proceeds through the call stack in the reverse order in which methods were called.
- If it finds an appropriate handler, then it passes the occurred exception to it. An appropriate handler means the type of exception object thrown matches the type of exception object it can handle.
- If the run-time system searches all the methods on the call stack and couldn't have found the appropriate handler, then the run-time system handover the Exception Object to the **default exception handler**, which is part of the run-time system. This handler prints the exception information in the following format and terminates the program **abnormally**.



Default exception handler message

```
Exception in thread "xxx" Name of Exception : Description  
... .. // Call Stack
```



Exception syntax

```
try {  
    // Block of code to try  
}  
catch(Exception e) {  
    // Block of code to handle errors  
}
```



Exception syntax

- A try block is the block of code (contains a set of statements) in which exceptions can occur; it's used to enclose the code that might throw an exception. The try block is always followed by a catch block, which handles the exception that occurs in the associated try block. A try block must be used within the method and it must be followed by a catch block(s) or finally block or both.
- The catch block catches and handles the try block exceptions by declaring the type of exception within the parameter. The catch block includes the code and it is executed if an exception inside the try block occurs. The catch block is where you handle the exceptions; so this block must be follow the try block.



Exception syntax

- The declared exception in catch block must be the parent class exception (i.e., Exception) or the generated exception type. However, the best approach is to declare the generated type of exception.
- In the Java language , you can use a try block without a catch block but you can't use the catch block without a try block
- *finally* defines a block of code we use along with the *try* keyword. It defines code that's always run after the *try* and any *catch* block, before the method is completed.
- **The *finally* block executes regardless of whether an exception is thrown or caught.**



Example

- ex

```
public class Main {  
    public static void main(String[ ] args) {  
        int[] myNumbers = {1, 2, 3};  
        System.out.println(myNumbers[10]); // error!  
    }  
}
```

```
public class Main {  
    public static void main(String[ ] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        }  
    }  
}
```



Finally block

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        } finally {  
            System.out.println("The 'try catch' is finished.");  
        }  
    }  
}
```



Methods to print Exception information

- **printStackTrace():** prints exception information in the format of the Name of the exception: description of the exception, stack trace.

```
import java.io.*;

class GFG {
    public static void main (String[] args) {
        int a=5;
        int b=0;
        try{
            System.out.println(a/b);
        }
        catch(ArithmeticException e){
            e.printStackTrace();
        }
    }
}
```

```
java.lang.ArithmeticException: / by zero
at GFG.main(File.java:10)
```

output



Methods to print Exception information

- **toString()** :prints exception information in the format of the Name of the exception: description of the exception

```
import java.io.*;

class GFG1 {
    public static void main (String[] args) {
        int a=5;
        int b=0;
        try{
            System.out.println(a/b);
        }
        catch(ArithmeticException e){
            System.out.println(e.toString());
        }
    }
}
```

```
java.lang.ArithmeticException: / by zero
```

output



Methods to print Exception information

- **getMessage():** prints only the description of the exception

```
import java.io.*;

class GFG1 {
    public static void main (String[] args) {
        int a=5;
        int b=0;
        try{
            System.out.println(a/b);
        }
        catch(ArithmeticException e){
            System.out.println(e.getMessage());
        }
    }
}
```

/ by zero

output



Throwing Exception

- When an exception is thrown using the **throw** keyword, the flow of execution of the program is stopped and the control is transferred to the nearest enclosing try-catch block that matches the type of exception thrown. If no match is found, the default exception handler terminates the program
- The **throw** keyword is useful for throwing exceptions based on certain conditions e.g. if a user enters incorrect data. It is also useful for throwing custom exceptions specific to a program or application
- The throw keyword in Java is used for explicitly throwing a single exception



Example of Throw

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int a,c,b;

        Scanner k=new Scanner(System.in);
        System.out.println("enter the first number");
        a=k.nextInt();
        System.out.println("enter the second number");
        b=k.nextInt();
        try {
            c=a/b;
            System.out.println("the result is"+c);

        }catch(ArithmeticException e) {
            System.out.println(e);
            e.printStackTrace();
        }

    }

}
```

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int a,c,b;

        Scanner k=new Scanner(System.in);
        System.out.println("enter the first number");
        a=k.nextInt();
        System.out.println("enter the second number");
        b=k.nextInt();
        try {
            if (b==0)
                throw new ArithmeticException(" don't divide by zero");
            c=a/b;

            System.out.println("the result is"+c);

        }catch(ArithmeticException e) {
            System.out.println(e);
            e.printStackTrace();
        }

    }

}
```



Custom exception

```
1
2 public class Minor extends Exception {
3     public Minor(String message) {
4         super(message);
5     }
6
7 }
8
```

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int age=0;

        Scanner k=new Scanner(System.in);
        System.out.println("enter your age");
        age=k.nextInt();

        try {
            if (age<18)
                throw new Minor(" your are under 18");

            else

                System.out.println("you can watch the movie");

        }catch(Minor e) {
            System.out.println(e);
            e.printStackTrace();
        }

    }
}
```



Throws keyword

- **throws** is a keyword in Java that is used in the signature of a method to indicate that this method might throw one of the listed type exceptions. The caller to these methods has to handle the exception using a try-catch block.
- **throws** keyword delegates the responsibility of exception handling to the caller (It may be a method or JVM) then the caller method is responsible to handle that exception.

```
type method_name(parameters) throws exception_list
```

exception_list is a comma separated list of all the exceptions which a method might throw.

Syntax of
java throws



Throws keyword example

```
1 package Handling;
2
3 public class Main {
4     public static int divide(int a, int b) throws ArithmeticException {
5
6         int result=a/b;
7
8         return result;
9     }
10    public static void main(String[] arg) {
11        int c=4;
12        int d=0;
13        try {
14            System.out.println("the result is"+divide(c,d));
15        } catch (ArithmeticException e) {
16            System.out.println("invalid division");
17        }
18    }
19 }
20 }
```



Chained Exceptions

- Chained Exceptions allows to relate one exception with another exception, i.e one exception describes cause of another exception.
- For example, consider a situation in which a method throws an `ArithmeticException` because of an attempt to divide by zero but the actual cause of exception was an I/O error which caused the divisor to be zero.
- The method will throw only `ArithmeticException` to the caller. So the caller would not come to know about the actual cause of exception. Chained Exception is used in such type of situations



Constructors of Throwable class

The 2 following constructors of throwable class support the chained exceptions in java

- `Throwable(Throwable cause)` :- Where cause is the exception that causes the current exception.
- `Throwable(String msg, Throwable cause)` :- Where msg is the exception message and cause is the exception that causes the current exception.

Methods Of Throwable class Which support chained exceptions in java :

1. `getCause()` method :- This method returns actual cause of an exception.
2. `initCause(Throwable cause)` method :- This method sets the cause for the calling exception.



Methods of Throwable class

Methods of Throwable class Which support chained exceptions in java :

1. `getCause()` method :- This method returns actual cause of an exception.
2. `initCause(Throwable cause)` method :- This method sets the cause for the calling exception.



Example of chained Exception

```
import java.io.IOException;
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int numerator=6;
        int c;
        String denominator;
        Scanner k=new Scanner(System.in);
        System.out.println("enter the denominator ");
        denominator=k.nextLine();
        try {

            c=Integer.parseInt(denominator);

            if (c==0) {
                ArithmeticException e=new ArithmeticException("you entered zero as denominator");
                e.initCause(new IOException ("is the cause"));
                throw e;
            }
            else{
                int z=numerator/c;
            }

        }catch(ArithmeticException e) {
            System.out.println("can't divide by zero"+e);
            System.out.println(e.getCause());
        }

        finally{
            k.close();
        }
    }
}
```



Try with resources

- The try-with-resources statement is a try statement that declares one or more resources.
- A resource is an object that must be closed after the program is finished with it.
- The try-with-resources statement ensures that each resource is closed at the end of the statement.
- Any object that implements `java.lang.AutoCloseable`, which includes all objects which implement `java.io.Closeable`, can be used as a resource.



example

```
InputStreams.java -- @FileReader.class
import java.io.*;
public class InputStreams {

    public static void main(String[] args) throws IOException {
        // TODO Auto-generated method stub
        // FileReader source=null;
        // FileWriter target=null;

        try (FileReader source=new FileReader("C:\\external\\daily\\RCA COURSES\\JAVA\\source.txt");
            FileWriter target=new FileWriter("C:\\external\\daily\\RCA COURSES\\JAVA\\destination.txt"))
        {

            int content;
            //reading source file and writing to target file character by character
            while((content=source.read())!=-1) {

                target.write((char)content);
            }
        }
    }
}
```

